



Fig. 5: Switching combinations program running

By default, GPIO-18 is high as it is connected to 3.3 volts from the board. When the push-to-on switch is pressed, it gets connected to the ground, thus making GPIO-18 low. This is the switching input, which is used for doing all other associated jobs with the Raspberry Pi.

GPIO-18 is wired for input signal, while all other GPIOs are connected as output signal. LEDs are connected to GPIOs as shown in Fig. 4. 470-ohm resistors are used to limit the current to LEDs. To avoid bouncing of switch, we have put a check of time delay for 2000 ms. For driving bigger loads, you can use relays or triacs in place of LEDs.

Working of the system is simple. Once the program is running on Raspberry Pi (explained in the software section), you will see the message "This is first combination" on the terminal window and LED1 starts blinking. Press the push-to-on button once. You will see the message change from "This is first combination"

to "This is second combination." Also, LED2 starts blinking in place of LED1. The same goes for the third and fourth combinations.

Software. With PHP and PHP-GPIO already installed, open the nano text editor with the below-mentioned command:

```
nano lightcontrol.php
where 'nano' is the text editor and
'lightcontrol' is the name of the project.
Now paste the below-mentioned
program in the nano text editor:
```

```
<?php
require_once('GPIO.php');
$button = 18; // pin no 12 is GPIO 18
$gpio = new GPIO();
$gpio->setup($button, "in");
$gpin1 = new GPIO();
$gpin1->setup(17, "out"); // pin no 11
                        is GPIO 17
$gpin2 = new GPIO();
$gpin2->setup(24, "out"); // pin no 18
                        is GPIO 24
$gpin3 = new GPIO();
$gpin3->setup(25, "out"); // pin no 22
                        is GPIO 25
$gpin4 = new GPIO();
$gpin4->setup(23, "out"); // pin no 16
                        is GPIO 23
$buttonstate = $gpio->input($button);
                        //initial state
$lightmode = 0;

while(true) {
    // beginning of the infinite loop
    $val = $gpio->input($button);
    usleep(2000); // bounce free
                                checking
    $val2 = $gpio->input($button);
    if($val == $val2) {
    if($val != $buttonstate) {
    if($val == 0) {
    if ($lightmode == 0) $lightmode = 1;
    elseif ($lightmode == 1) $lightmode
                                = 2;
    elseif ($lightmode == 2) $lightmode
                                = 3;
    elseif ($lightmode == 3) $lightmode
```

```
                                = 0;
    }
    $buttonstate = $val;
    }
    }
    if ($lightmode == 0){
    echo "This is first combination:\n";
    $gpin4->output(17,1); // making gpio
                                17 high
    usleep(500000); // holds the state
                                for 0.5 second
    $gpin4->output(17,0); // making gpio
                                17 low
    usleep(500000); // holds the state
                                for 0.5 second
    }
    if ($lightmode == 1){
    echo "This is 2nd combination:\n";
    $gpin1->output(24,1);
    usleep(500000);
    $gpin1->output(24,0);
    usleep(500000);
    }
    if ($lightmode == 2){
    echo "This is 3rd combination:\n";
    $gpin2->output(25,1);
    usleep(500000);
    $gpin2->output(25,0);
    usleep(500000);
    }
    if ($lightmode == 3){echo "This is
                                forth combination:\n";
    $gpin3->output(23,1);
    usleep(500000);
    $gpin3->output(23,0);
    usleep(500000);
    }
    }
    ?>
```

Save it by pressing 'control+o' and then exit by 'control+x.' Move this file to '/var/www,' from where it is executed.

Now on the Raspberry Pi terminal window, issue the following command:

```
sudo php -f /var/www/lightcontrol.php
```

You will see the program running as shown in Fig. 5. Every time you press the switch, the message will change and the corresponding LED will blink. ●

The author is a mechanical engineer working as an additional general manager at NTPC Limited